

Shopping Copilot

Business proposal for an apparel shopping assistant

SMYA | English edition | 27 September 2026

01 Executive summary

Shopping Copilot is a product selection assistant for small and medium-sized apparel retailers. We propose a controlled pilot within one retailer's limited catalog, helping shoppers turn an imprecise need into an evidence-backed shortlist while reducing repetitive product searches, comparisons and preference clarification by service staff. The first priority is faster selection, lower manual workload and accurate product information; sales growth is a commercial hypothesis to validate through an experiment.

Business problem and proposed response

Shoppers repeatedly move between search results, product pages and service conversations. Changing color, material or budget can require them to explain their needs again. Shopping Copilot combines conversation, product results, a requirements summary and comparison of up to three saved products. It separates requirements, preferences and exclusions, supports revisions and undo, and explains recommendations and unknowns using catalog evidence. The shopper confirms the final selection. [S2–S5]

Proposed pilot decision

Approve an eight-week staged pilot: two weeks for baseline and data preparation, two weeks for deployment hardening, and four weeks of limited-traffic evaluation. The planning allowance is S\$3,500 for setup and up to S\$600 per month for operations. Reserving two months of operating costs gives a total authorization ceiling of S\$4,700. These are planning assumptions requiring owner approval against actual quotations and effort. Public traffic is conditional on acceptance of real data, human assistance, security controls and rollback capability.

Expected outcomes and current maturity

Pilot targets are a reduction in median time to a usable shortlist from an assumed 10 to 6 minutes, a 40% reduction in repetitive selection workload, and 98% evidence accuracy for sampled objective claims. Conversion growth from 3.0% to 3.3% remains a hypothesis. The prototype supports a synthetic-catalog demonstration and a separate real-search adapter. Local search data, index and model files are present; their presence alone does not establish readiness or data quality. Confirmation records a session decision, not an order or payment. [S3–S7]

02 Problem and opportunity

Target business and stakeholders

The initial use case is a Singapore-based apparel SME serving online shoppers in English. The pilot starts with categories such as dresses and T-shirts, using retailer-authorized product information. The business owner cares about sales and operating costs; service staff about handling time and repeated explanations; merchandising staff about attributes, prices and inventory quality; and shoppers about understanding differences and making a reliable choice. This is a pilot business profile, not evidence of a signed customer or completed interviews.

Repeated work in the current journey

After a keyword search, the shopper opens individual pages and manually compares material, fit and price. Uncertainty leads to a service enquiry. Staff repeat questions about needs, consult product records and assemble options. A changed requirement can restart that work. The potential consequences are capacity constrained by staff time, interrupted purchase decisions and inconsistent descriptions across employees. The pilot will establish whether these frictions exist rather than presume a measured revenue loss.

Baseline assumption	Value	Validation method
Eligible website sessions per month	10,000	Analytics excluding bots and internal traffic
Repetitive selection enquiries per month	600	Ticket classification and sample review
Staff handling time per enquiry	6 minutes	Timed observation including follow-up
Median time to usable shortlist	10 minutes	Matched user tasks
Purchase conversion / order value	3.0% / S\$55	Linked retailer order and session data

Under these assumptions, repetitive enquiries consume 60 staff hours per month. Recovered time could support complex presales and aftersales work without reducing headcount. During weeks 1–2, interview the owner, two service staff and 5–8 shoppers, and review at least 100 enquiries. Replace the assumptions with measured baselines before confirming pilot economics.

03 Solution and shopper journey

Shopping Copilot organizes product selection as a revisable, explainable conversation. It chooses whether to clarify, retrieve, answer a product question, compare or update the shortlist according to session state; not every message triggers another search. The implementation combines rules, structured state and retrieval tools. Model assistance is optional and off by default, so the prototype should not be described as an autonomous sales agent using a generative model on every turn.

[S3–S6]

Step	Shopper experience and system behavior
1 Express a need	For example, “I need a blue dress. I prefer cotton.” The assistant separates explicit requirements from preferences and shows the recorded needs.
2 Review and clarify	Show candidates and supporting evidence; ask a bounded clarification when useful. Unlisted information remains unknown.
3 Revise and refine	Revise requirements, hide an item, request more options or undo a change; review the updated requirements before continuing.
4 Save and compare	Save up to three products within the session; the frontend comparison button becomes available with at least two saved options. Review attributes and evidence.
5 Confirm a decision	The shopper confirms. Later preference or shortlist changes can invalidate confirmation and require review. Transactions remain in the retailer’s existing process.

How the functions work together

A message first resolves the shopper’s intent and any reference to the visible products. The workflow applies explicit requirement changes, then chooses a bounded next action: clarify, search, answer, compare, save or confirm. “How much is #2?” reads the current item without searching again; “Blue instead” changes the active requirement and refreshes eligible results. Undo restores the prior state and relevant view. This continuity is the mechanism intended to reduce repeated explanation and page checking.

For example, “I need a blue dress. I prefer cotton” records dress and blue as requirements and cotton as a preference. Retrieval finds candidates; catalog checks enforce requirements, while supported cotton matches can move ahead without excluding all alternatives. “Compare #1 and #2” binds those numbers to stable product IDs before comparison. The shopper can revise the preference, compare again and explicitly finalize a saved shortlist. Unknown price or stock stays unknown throughout. [S3–S6]

Scope and subsequent work

Existing scope includes session requirements, catalog retrieval and reranking, evidence displays, product questions, saving, comparison, explicit confirmation and audit export. Search currently explores one product category at a time; deferred category details can be remembered for later. Pilot work adds retailer data acceptance, protected deployment, human assistance and business-event/order attribution. Accounts and cross-session personalization remain future work. Live web-wide price comparison, stock reservation, guaranteed fit, payment, refunds and autonomous ordering are outside scope. [S2–S8]

04 Business value and adoption

Value	How value is created	Outcome to measure
Productivity	Reuse session preferences and candidates to reduce repeated questions and page checking	Staff minutes per completed selection task
Service	Revise needs, inspect evidence and compare within one journey	Time to shortlist and satisfaction
Revenue opportunity	Help shoppers resolve uncertainty and proceed to the existing purchase journey	Purchase conversion by group and incremental gross profit
Risk control	Keep unknowns explicit and recheck decisions after changes	Factual error rate and misleading claims
Scale	Separate catalog maintenance from enquiry handling and reuse workflow/data standards	Peak service quality and cost per session

Differentiation and alternatives

Standard filters serve shoppers with precise criteria; human staff handle complex and exceptional cases; FAQ tools answer fixed policy questions. Shopping Copilot targets the stage between an uncertain need and an explainable shortlist by retaining current requirements, supporting changes of mind and showing evidence. The pilot compares it with the retailer's existing search and service journey rather than assuming superiority over every alternative. Better filters may be more economical when catalog attributes are sparse or needs are simple.

Adoption and future commercialization

Begin as a single-retailer website component or dedicated selection page, with staff training and an exception process. Assign one business owner for outcomes, one merchandising owner for data and one technical owner for service quality, with weekly feedback reviews. After value is demonstrated, consider a managed subscription with a usage allowance for other SMEs. Pricing must cover onboarding, operations and support and be validated through willingness-to-pay interviews. No subscription revenue is assumed at this stage, and paid placement is not part of relevance ranking.

05 Impact metrics and validation design

These are proposed targets. Definitions, denominators and exclusions must be fixed before the pilot begins. Passing software tests does not establish achievement of business KPIs. [S9]

Metric and owner	Baseline to target	Measurement definition
Time to shortlist / Product owner	Assumed 10 → 6 minutes	From initial need to at least two candidates meeting verified hard requirements; also report task completion to avoid survivor bias
Manual workload / Service lead	Assumed 60 → 36 hours/month	Normalize for enquiry volume and mix; include escalations and rework
Evidence accuracy / Merchandising	Measure baseline → ≥98%	Review at least 200 checkable claims; fraction both correct and supported
Purchase conversion / Business owner	Assumed 3.0% → 3.3%	Purchasing sessions / eligible randomized sessions; +0.3 percentage points, or +10% relative
Satisfaction and reliability / Product and Tech	CSAT ≥4/5; p95 ≤10 seconds	Report survey response rate; ≥99% successful interactions, plus errors and unknown-data rates

Observability and technical evaluation

Per-turn receipts expose the chosen action and reason, requirement changes, result IDs, evidence and model usage. Audit export links execution records with digest checks; it helps inspect sequence consistency but is not a tamper-proof external ledger. Comparison also reports status and latency. Reviewers can follow why a clarification, retrieval or confirmation occurred through structured decisions rather than private model reasoning. Production retention, access controls and operational dashboards remain pilot work. [S3, S5]

Golden-path evaluation follows a need through retrieval, correction, Undo, saving, comparison and explicit confirmation. Failure cases cover empty results, unpriced strict budgets, invalid product references, stale state, search failure, partial comparison and changes after confirmation. Before release, extend adversarial cases with instructions embedded in product text, fabricated evidence, cross-session requests and repeated or oversized input. Check factual support, requirement preservation, recovery and absence of unintended confirmation. Record elapsed task time and manual steps against the same baseline task; automated checks alone cannot establish business savings. [S9]

Experiment and attribution

Take 20% of eligible traffic for the pilot and randomize it between the existing journey and a journey with assistant access, keeping assignments stable. Analyze by initial assignment, retaining non-users in the treatment denominator. Add pseudonymous experiment IDs and search, comparison, confirmation, escalation and purchase events. Purchases must come from the retailer's order system; shortlist confirmation is not a purchase. In weeks 1–2, use at least 20 shoppers for counterbalanced task testing and report time medians and distributions.

At 10,000 eligible sessions per month, the four-week pilot yields about 1,000 sessions per arm, generally insufficient to detect a change from 3.0% to 3.3%. A two-sided 5% test with 80% power and independent observations requires approximately 53,000 sessions per arm; repeat visitors may increase the requirement. The four-week decision therefore prioritizes workflow quality and safe operation. Report conversion intervals and direction, then extend sampling before claiming revenue impact.

06 Budget and economics

The following monthly scenarios assume one retailer and full coverage of eligible traffic, in Singapore dollars. They are not a forecast of pilot revenue: the pilot has limited traffic while setup and operating costs still occur. Assumptions are a loaded staff cost of S\$25/hour, S\$55 order value and 40% merchandise gross margin. Additional marketing, payment, returns and other variable expenses are not yet deducted.

Budget item	Planning allowance
One-time implementation: 100 hours × S\$25	S\$2,500
Data preparation, acceptance and training: 24 hours × S\$25	S\$600
One-time contingency	S\$400
Total one-time budget	S\$3,500
Monthly compute and storage / model allowance / monitoring / maintenance	S\$200 / 150 / 100 / 150
Total monthly operating budget	S\$600

Base case: released time = 600 enquiries × 6 minutes ÷ 60 × 40% = 24 hours, worth S\$600 in capacity. Incremental orders = 10,000 × (3.3% – 3.0%) = 30; incremental sales are S\$1,650 and merchandise gross profit is S\$660. Monthly net benefit including capacity is 600 + 660 – 600 = S\$660. The simple economic payback on setup is about 5.3 months.

Scenario	Workload cut / conversion	Monthly net incl. capacity	Monthly net, unchanged payroll
Conservative	20% / 3.0%	–S\$300	–S\$600
Base	40% / 3.3%	S\$660	S\$60
Upside	50% / 3.6%	S\$1,470	S\$720

With unchanged payroll, S\$600 of capacity is not cash saved. The base case leaves only S\$60/month toward setup, implying about 58.3 months of simple payback before omitted expenses, which would reduce returns further. Covering S\$600 of operating cost through incremental gross profit alone requires about 28 additional orders, or a 0.28-percentage-point conversion increase. Expansion should depend on usable recovered capacity and validated profit, not treating theoretical capacity as cash.

07 Technical feasibility and scaling

Architecture and reasoning loop

Shopping Copilot uses a stateful Python orchestrator with specialist modules for intent, memory, retrieval, ranking and comparison. Its planning pattern is an adaptive policy loop: interpret the message and current context, validate a state update, choose a permitted action, execute the relevant tool, inspect its result, and return evidence with the next available choice. Candidate evidence can justify a clarification; a failure can produce a recoverable response. The loop advances through shopper turns rather than running an unrestricted autonomous task. Generative assistance is optional and off by default. [S3, S6]

End to end: shopper message → intent and reference resolution → versioned requirements → policy decision → search or a read-only answer or comparison → evidence and visible options → shopper feedback → updated state. Saving and finalization are explicit branches. Deterministic controls retain responsibility for state mutation and action boundaries even when a model assists interpretation.

Intent recognition and requirement updates

The intent router and control handlers distinguish browsing, directed search, comparison, product questions and commands such as Undo or Start over. Catalog-aware extraction separates hard requirements, weighted soft preferences and exclusions. Separately phrased questions and edits can share a turn; product references bind to the pre-update display. Ambiguous categories or item references request clarification rather than silently selecting an interpretation. This produces an actionable request while preserving what the shopper actually committed to. The language coverage is bounded, not unrestricted understanding. [S3, S6]

Explicit state and memory

Session memory stores the active category, hard conditions, weighted preferences, exclusions, shown and rejected product IDs, pending questions and deferred category details. Source turns and evidence make requirement changes inspectable. Snapshots and history support Undo and Redo, including the relevant visible results; saved products use stable IDs rather than changing screen ranks. A session ID, turn and state version travel with retrieval results so stale or cross-session results can be rejected. The effect is continuity through corrections without replaying the whole conversation. [S3, S6]

The service keeps session data in memory, with defaults of 128 sessions and a 3,600-second idle lifetime. These are storage limits, not measured concurrent-user capacity. The proposal claims session memory only. Durable cross-device memory, account ownership and shared storage require additional implementation.

Catalog retrieval and contextual ranking

The search adapter constructs an English query from the structured requirements and calls `search_products(query, top_k)`. The shared search tool retrieves lexical candidates with BM25, then scores query–product pairs using a local sequence-classification model. `get_product_details(product_ids)` supplies listing fields for those IDs. The adapter validates the returned IDs and finite scores, checks supported hard conditions and exclusions, and favors unseen items when more results are requested. The interface shows at most ten products. [S4]

Catalog-supported soft preferences for attributes such as color, material or fit can adjust the final order without becoming hard exclusions. The underlying search scores remain relevance scores, not probabilities, quality guarantees or purchase confidence. When evidence is missing, the system does not invent a match. The current adapter leaves price and ratings unknown, so a strict budget

cannot be verified against those records. An empty pool prompts an explicit revision or recovery choice; constraints are not silently relaxed. The intended effect is a smaller, relevant and explainable set of options. [S3, S4]

Evidence based clarification and product questions

The adaptive policy uses distinctions in retrieved candidates to decide whether an optional question could meaningfully narrow the choice. Its attention limits reduce repeated questioning; explicit browsing or a request to see results can skip optional refinement. A missing searchable category or unverifiable strict budget requires a different clarification. Questions about a displayed item use its catalog record and preserve the current page and open choice. Unsupported attributes are reported as unknown. This keeps guidance useful while avoiding unnecessary searches and forced questionnaires. [S3]

Description and comparison

The description adapter passes up to three resolved products and current requirements to DescriptionComparison. With a configured model provider, the pipeline extracts category-relevant attributes, creates an objective comparison, and then adds personalized suitability reasons and cautions. Personalization is skipped when no requirements or preferences exist. Source-quote checks reject unsupported quotations; inferred values remain labeled. Objective facts are not rewritten to fit a preference, and comparison preserves the requested product order. [S5]

Without a provider, offline_preview exposes direct catalog facts and unknowns, without generated personalized advice. A failed model stage yields partial output; an explicit retry can resume the comparison. The session cache reuses completed comparisons when product IDs and order, requirements, supplied facts and model identity are unchanged. Changed inputs invalidate the cache. This avoids unnecessary repeat calls while keeping updated comparisons aligned with the shopper's current needs.

Typed tool boundaries and module integration

RetrievalResultV2 and RankingResultV2 define versioned contracts for session context, candidates and ranking. Validators check identity, candidate limits, continuous ranks and matching state context. The selection handoff uses show-me-your-agent.selection.v1; comparison returns description-comparison.v1 with product profiles, comparison values, evidence references, status and usage. Empty, duplicate or oversized selections are handled explicitly. These contracts make handoffs inspectable and reduce stale-data and malformed-output errors. [S3–S5]

Platform and tooling usage

The implementation uses Python modules and explicit orchestration rather than a framework-managed team of autonomous LLM agents. BM25, PyTorch and Transformers serve local search; JSON interfaces connect the service and shopper UI. A configurable provider supplies optional local or cloud model assistance. Adapters connect the team's search and description modules without requiring their internals to share conversation state. This separation supports independent module testing and a consistent state authority. [S3–S6]

Prerequisites and scaling

The retailer must supply authorized records with stable IDs, source fields, price currency, timestamps and variant or stock evidence. The local search table, index and model files exist, but the review runtime lacks several search dependencies; live retrieval was not rerun in this review. A pilot needs a configured runtime and real-query acceptance, not just a passing health check. The current catalog mapping also needs real price and inventory integration. [S4, S7]

Before public traffic, add session ownership, rate limits, production API routing, shared state, per-session concurrency controls, timeouts and monitored fallback. The prototype HTTP service and serialization are not proof of production throughput. Measure latency and token usage separately for search, model stages and cache hits; load-test at twice the retailer's measured peak. Additional categories need schema and evidence tests; additional retailers need tenant isolation. These are deployment requirements, not existing capacity claims. [S8]

08 Risk management and human oversight

Automation is bounded to helping the shopper form an informed choice. The shopper retains decision authority; the retailer remains responsible for product facts, special commitments, transactions and exceptions. The existing confirmation step is shopper confirmation, not staff approval, live-agent transfer or integration with a service ticket. [S2–S3]

Risk	Existing control	Control to add before pilot
Incorrect or excessive claims	Evidence and unknown states; comparison separate from confirmation	Fact sampling; no personal-fit guarantee; suspend affected features after serious misleading claims
Stale or missing data	Fallback for unknown prices and ratings	Assign refresh ownership/cadence, label stale values and confirm in the commerce system
Lost state or repeated actions	Serialized frontend requests, no automatic POST replay, bounded sessions	Shared state, version checks, idempotency and post-timeout status lookup
Privacy and unauthorized access	Backend-only model secrets; operations limited to products shown in-session	Session ownership, data minimization, redacted logs, access control and retention limits
Malicious text or model failure	Bounded tool scope; model-off operation	Treat catalog/input as untrusted data; adversarial tests, rate limits and safe fallback

Autonomy and approval checkpoints

The agent can retrieve, rank, display evidence and propose useful questions within the session. The shopper controls preference changes, saved options and finalization; a generic acknowledgement or conditional instruction must not be treated as permission to finalize. Edits can invalidate a previous confirmation. No purchase, payment or stock reservation tool is available. The demo must show this real confirmation boundary. Staff escalation for exceptional or commercially binding decisions belongs to the pilot workflow below and is not an implemented approval service. [S3]

Safety and security boundaries

The description prompt explicitly treats product text as untrusted data, and the pipeline checks supporting quotes against supplied product fields. Typed inputs, bounded product selection and limited tool access constrain what model output can affect. These controls reduce exposure but do not prove prompt-injection immunity or correct interpretation of every quote. Before deployment, test malicious catalog instructions and enforce least privilege through backend credentials, session ownership, authorized catalog access and protected logs. Cloud model use must disclose that messages and selected catalog excerpts may leave the local service; avoid sending unnecessary personal information. [S3, S5, S6]

Human assistance workflow

Add a clear “Contact a person” route before the pilot. Triggers include an explicit request, failure after two clarification attempts, unverifiable hard requirements, conflicting product data and out-of-scope order, payment or return questions. With the shopper’s confirmation, send only the necessary requirements summary, candidate products and unresolved questions to staff; do not default to a full transcript or sensitive data. Propose a 15-minute first response during staffed hours and display actual hours otherwise. This route, routing logic and service commitment need implementation and acceptance; live handoff is not present in the current code.

The technical owner monitors errors, p95 latency, session cost and data freshness; merchandising reviews recommendation evidence weekly; the business owner controls suspension and expansion. Unauthorized data access, unintended transaction execution or serious misleading claims trigger immediate suspension of the affected function and fallback to the existing journey, retaining necessary incident evidence.

09 Implementation plan and acceptance

Stage	Work and owner	Gate to proceed
Weeks 1–2 Baseline and data	Business: enquiry/sales baseline. Merchandising: authorized catalog. Product: interviews and task tests.	Agreed category, field coverage, sources and measured baseline; revised budget and KPIs
Weeks 3–4 Pilot hardening	Tech: search assets, routing, session protection, monitoring and events. Service: human assistance.	Real-query and end-to-end acceptance; timeout, access, unknown-data, rollback and human-help checks pass
Weeks 5–8 Controlled evaluation	Product: randomized allocation. Business: weekly workflow/cost review. Merchandising: evidence checks.	Quality, safety and cost gates met; conversion uncertainty reported; continue, revise or stop decision
Week 9 onward Conditional expansion	Increase traffic within measured capacity; continue conversion experiment and expand category by category	No degradation at added load; economics reassessed using observed data

Pilot acceptance criteria

Continue if median selection time reaches 6 minutes without lower completion than control; normalized manual workload falls 40%; evidence accuracy reaches 98%; no severe privacy, access or misleading-claim incident occurs; interaction success is at least 99%, p95 latency at most 10 seconds, and monthly operating cost at most S\$600. If targets are missed, diagnose data, interface or model causes, revise and retest. A favorable metric cannot compensate for a severe risk.

Handover and exit

Deliver a verified catalog/index version, deployment configuration, operating and staff-training guides, monitoring dashboard, metric definitions and pilot evaluation. The business owner approves each next stage; the technical owner must be able to disable assistant access and restore the original journey. If essential data, human assistance or security controls are not ready by week 4, retain internal demonstration mode and revise the schedule rather than expose the incomplete prototype to public traffic.

10 Demo script and evidence alignment

Use a 5–6 minute demo with one business story: turning a changing shopping need into an evidence-backed shortlist. Show the synthetic Demo catalog label and start a new session. Keep a visible timer and show the same task in the baseline journey when comparing manual steps. Use the configured interface’s equivalent controls or chat commands. A second, explicitly labeled model-enabled segment is needed to demonstrate generated personalized advice. [S2, S3]

Time	Action and display	Claim supported
0:00–0:40	Show the existing search, page-review, enquiry and comparison journey; identify assumed baselines	Clear problem; no claim of measured savings yet
0:40–1:30	Enter “I need a blue dress. I prefer cotton.” Show products, Must have and Preferences	Structured needs and catalog recommendations
1:30–2:20	Change a color or preference, show the updated summary, then enter “Undo”	Visible, reversible requirement changes
2:20–3:30	Open details and evidence, save two products, compare, and reveal unlisted attributes	Evidence-backed comparison and disclosure of unknowns
3:30–4:20	Confirm the selection and show the no-order notice; remove one item and show confirmation invalidation	Shopper control and review after changes
4:20–5:30	Explain plans for the real catalog, human-help route and deployment; show KPIs and budget	A path from prototype to a measurable pilot

Boundaries of demo evidence

A timed demonstration shows the observed task duration, tool sequence and approval point; it does not establish a population-wide 40% reduction. Show requirement state before and after a correction, the resolved comparison IDs, an unknown attribute, and the finalization record. Demonstrate one invalid reference or unpriced strict-budget recovery without inventing data or silently relaxing requirements. Keep staff handoff labeled as planned. With the model off, describe comparison as an offline facts preview, not validated generative reasoning. [S3–S6]

11 Evidence and assumption register

References identify the local project modules reviewed for this proposal. S1 maps the judging requirements; S2–S8 support architecture and capability statements; S9 states the verification boundary. Business baselines, budgets, growth figures and service commitments remain planning assumptions or targets.

S1 Supplied judging slides and business proposal guidelines

The judging slide covers goal and scope, architecture and reasoning, tools, human oversight, safety, observability and platform usage. The Consistent Message slide requires proposal claims to match demo evidence. These map to Sections 1–3, 7, 8, 5 and 10.

S2 `agentic_workflow/SUBMISSION_EN.md` and `DESCRIPTION_INTEGRATION.md`

Documented shopper journeys, limitations and integration behavior; checked against the runtime and adapters. Historical verification counts are not new measurements.

S3 `agentic_workflow/mvp/server.py`; `shopping_agent/agent.py`; `shopping_agent/policy.py`; `mvp/audit.py`

Orchestration, clarification, product references, session limits, selection, confirmation and execution records. Paths after the semicolon are relative to `agentic_workflow`.

S4 `search_tool/tool.py`; `agentic_workflow/shopping_agent/search_adapter.py`;

`agentic_workflow/retrieval-and-reranking/techjam_agent/contracts_v2.py`

BM25, local reranking, details lookup, catalog filtering, soft-preference ordering and versioned contracts.

S5 `description_module/pipeline.py` and `README.md`;

`agentic_workflow/shopping_agent/description_adapter.py`

Extraction, objective and personalized comparison, quote checks, statuses, handoff and caching integration.

S6 `agentic_workflow/intent-recognition/intent_router`; `conversation-state-memory/src/state_memory`; `shopping_agent/requirement_enhancer.py`; `shopping_agent/model_provider.py`; `__main__.py`

Intent, structured memory and optional model configuration. Paths after the semicolon are relative to `agentic_workflow`.

S7 `agentic_workflow/preflight.py`; `search_resources.py`; `search_tool/artifacts` and `search_tool/models`

Local resource inspection found the product table, retrieval index and saved model. Preflight in the review runtime reports missing `bm25s`, `torch`, `transformers` and `pyarrow` dependencies; this is not evidence that all user environments lack them.

S8 `agentic_workflow/mvp/server.py` and deployment requirements in this proposal

The local service is a prototype. Authentication, shared state, hardened hosting, human assistance and retailer order attribution require implementation and acceptance. Separate frontend build claims are outside this review.

S9 Verification for this revision

Description module: 8 automated tests passed with simulated responses. Workflow regression suite: 599 tests passed. Search dependency preflight was run; no live-model quality, retailer-data acceptance or business-outcome measurement was performed.

Verification boundary: offline and simulated tests establish covered behavior only. They do not establish retailer-data accuracy, live-model quality, prompt-injection immunity, concurrent capacity or business returns. Synthetic demonstration data must remain visibly identified; live-search and cloud-model acceptance require a separately configured run.